Azure DevOps: identity, network, services và pipeline

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Resource group, identity và mạng

Azure tổ chức tài nguyên theo tenant, subscription, resource group và resource. Tenant liên quan identity; subscription là phạm vi billing/quản trị; resource group nhóm tài nguyên có vòng đời phù hợp. Đặt một ứng dụng trong resource group không tự khiến các tài nguyên có cùng quyền hoặc cùng network. Cần xem RBAC, VNet, route và NSG riêng.

Microsoft Entra ID xác thực identity; Azure RBAC quyết định identity có thể thực hiện hành động nào trên scope nào. Managed identity là identity gắn với tài nguyên Azure, giúp ứng dụng lấy token mà không lưu password client trong cấu hình. Ví dụ App Service cần đọc secret ở Key Vault: cấp managed identity cho App Service và quyền đọc đúng secret. Nếu lỗi 403, kiểm tra identity thực tế, scope/role hoặc access policy, network restriction và log; đừng gắn quyền Owner toàn subscription để thử.

VNet là mạng riêng logic; subnet chia địa chỉ; NSG lọc traffic theo rule. Private Endpoint đưa một dịch vụ Azure vào VNet qua IP private. Tuy nhiên, private endpoint cần DNS trỏ đúng private IP; tạo endpoint mà client vẫn phân giải tên public thì đường đi có thể không như mong muốn. Private endpoint cũng không tự bảo đảm đường public đã bị tắt; cần xem cấu hình dịch vụ đích. Đây là lỗi phỏng vấn và vận hành phổ biến vì có nhiều lớp cùng tham gia.

Bài tập luồng truy cập

Vẽ browser -> App Service -> Azure SQL. Cho người dùng truy cập App Service qua HTTPS; App Service vào SQL qua private endpoint và managed identity. Ghi ở mỗi mũi tên: DNS, IP public/private, TLS, identity và quyền. Giả sử SQL trả lỗi truy cập: phân biệt lỗi DNS/route/NSG với lỗi RBAC/database authorization bằng tín hiệu nào? Một kết nối TCP thành công không chứng minh truy vấn được phép.

Dịch vụ ứng dụng và dữ liệu

App Service chạy ứng dụng web/API theo mô hình managed; bạn vẫn quản lý mã, cấu hình, identity và release. Azure Functions phù hợp các handler theo sự kiện hoặc yêu cầu, nhưng cần hiểu trigger, timeout, cold start và khả năng chạy lại. Storage có nhiều loại như blob/file/queue theo mục đích. Azure SQL là database quan hệ managed; Service Bus là broker cho message/queue/topic. Chọn dịch vụ bằng yêu cầu: request đồng bộ hay công việc nền, dữ liệu quan hệ hay object, độ trễ, durability và cách phục hồi.

Ví dụ ứng dụng nhận file: API ghi metadata vào Azure SQL, file vào Blob Storage, phát message sang Service Bus để xử lý nền. Worker Azure Function nhận message, tạo thumbnail và cập nhật trạng thái. Một message có thể được giao lại khi xử lý thất bại; handler cần tính đến idempotency để tránh tạo kết quả trùng. Nếu Storage không sẵn sàng, API cần trả lỗi có ý nghĩa và không ghi metadata giả rằng file đã được lưu.

Key Vault giữ secrets/keys/certificates; không phải nơi thay thế mọi file cấu hình. Cấu hình không nhạy cảm có thể ở app settings hoặc hệ quản lý cấu hình phù hợp; secret cần quyền hẹp, rotation và audit. Khi app lỗi sau rotation, hỏi token/secret nào, phiên bản nào, cache khi nào hết hạn và app đã reload

chưa.

Lab chi phí thấp theo từng bước

Trước tiên chỉ dùng Microsoft Learn sandbox hoặc sơ đồ nếu bạn chưa có Azure subscription/budget. Nếu có tài khoản thử, đặt budget/cảnh báo, xác nhận subscription và region, tạo resource group lab có tên riêng. Tạo **một** dịch vụ đơn giản theo tutorial chính thức, ghi ID, SKU, giá dự kiến và cách truy cập; kiểm tra rồi xóa resource group lab sau khi đã chắc chắn nó không chứa dữ liệu khác. Sau đó mới mở rộng tới private endpoint/SQL, vì nhiều dịch vụ kết hợp có thể tạo phí đáng kể. Ghi hóa đơn thực tế sau lab để hiệu chỉnh ước tính.

Azure DevOps Pipelines và release governance

Azure Pipelines định nghĩa stages, jobs và steps trong YAML. Stage có thể là build, test, deploy staging, deploy production. Mỗi stage cần đầu vào/đầu ra và điều kiện rõ. Pipeline nên build artifact một lần từ commit cụ thể, kiểm tra, rồi promotion cùng artifact qua môi trường. Service connection cấp quyền để pipeline nói chuyện với Azure; bảo vệ quyền này, tách môi trường và không để pull request chưa tin cậy dùng credential production.

Branch policy và pull request review giúp kiểm soát thay đổi vào nhánh phát hành. Environment approvals cho phép người có trách nhiệm xem bằng chứng trước thay đổi rủi ro. Approval không thay test: người duyệt cần biết commit, artifact, kết quả test, scan, plan IaC, tác động và rollback. Release notes nên mô tả thay đổi người dùng/hạ tầng/dữ liệu, không chỉ liệt kê commit message.

Bài tập thiết kế pipeline

Viết YAML giả lập theo bốn stage: validate (lint/unit test), build (image/artifact), deploy-test (smoke/integration), deploy-prod (approval, rollout, kiểm tra sau triển khai). Vẽ đường đi của secret và service connection. Nếu integration test ở test fail, production stage phải làm gì? Nếu deployment production thành công nhưng smoke fail, ai nhận alert và cách rollback? Đừng tạo connection cloud thật chỉ để hoàn thành bài tập giấy.

Bicep, ARM và Terraform trong Azure

ARM là nền tảng quản lý/deploy tài nguyên Azure; ARM template dùng JSON. Bicep là ngôn ngữ khai báo gọn hơn cho Azure, biên dịch thành ARM template. Terraform dùng state riêng và provider để quản lý Azure cùng các nền tảng khác. Không nên gọi Bicep là "Terraform của Microsoft" rồi bỏ qua khác biệt state, lifecycle và cách preview. Khi làm việc trong tổ chức, lựa chọn còn chịu ảnh hưởng governance, kỹ năng đội ngũ và hệ thống hiện có.

Bicep có what-if để xem thay đổi dự kiến; Terraform có plan. Cả hai cần review và phân quyền. Không đặt bí mật trong file tham số được commit. Nếu một tài nguyên được quản lý bởi cả hai công cụ hoặc sửa tay thường xuyên, drift và quyền sở hữu trở nên mơ hồ. Quy định rõ tài nguyên nào do pipeline nào quản lý.

Bài tập so sánh

Lấy một resource group và một storage account làm ví dụ. Viết bảng: nơi khai báo, cách preview, trạng thái lưu ở đâu, ai apply, cách xử lý rollback và drift. Không cần thuộc cú pháp hết; mục tiêu là hiểu quy trình thay đổi. Sau đó đọc một mẫu Bicep chính thức và một mẫu Terraform Azure, chỉ ra resource, parameter/variable và output.

Application Insights, Log Analytics và KQL

Application Insights theo dõi ứng dụng: request, dependency, exception và trace tùy cấu hình instrumentation. Log Analytics Workspace lưu/truy vấn log Azure Monitor; KQL là ngôn ngữ truy vấn để lọc, nhóm và tính số liệu. Metrics giúp thấy xu hướng nhanh; log và trace giúp điều tra chi tiết. Luôn biết nguồn dữ liệu, sampling/retention và chi phí ingest. Nếu không có telemetry từ app, dashboard chỉ nhìn hạ tầng sẽ bỏ sót lỗi business.

Ví dụ sau release, 5xx tăng ở endpoint POST /notes. Truy vấn cần lọc đúng ứng dụng, thời gian và operation; nhóm theo version hoặc dependency để xem lỗi mới có gắn với release không. KQL có toán tử lọc, project và summarize; khi học, viết câu hỏi bằng tiếng Việt trước, rồi mới chuyển thành truy vấn. Một truy vấn trả 0 dòng có thể do không có lỗi, cũng có thể do sai bảng/time range hoặc telemetry chưa được gửi.

Bảo mật ứng dụng và tự động hóa có kiểm soát

JD 2 còn yêu cầu OWASP Top 10, threat modeling, SAST/DAST/SCA, CVE và least privilege. Với ví dụ ứng dụng tải file vào Blob Storage rồi phát message xử lý nền, tài sản gồm file người dùng, metadata, credential và token; điểm vào gồm API, Service Bus message, storage event và pipeline. Hỏi theo STRIDE: ai có thể giả mạo người dùng/message; dữ liệu có thể bị sửa; hành động có audit; file có bị lộ; hệ thống có thể bị nghẽn; quyền có bị leo thang. Với mỗi rủi ro, ghi một biện pháp và phép kiểm tra: giới hạn loại/kích thước file, token đúng audience, quét dependency, quyền Storage tối thiểu và log không chứa secret.

SAST tìm vấn đề trong source; SCA tìm dependency có lỗ hổng đã biết; DAST thử ứng dụng đang chạy. CVE cần triage theo phiên bản thực dùng, đường code, cấu hình, độ phơi bày và bản vá. Quy trình ghi mức ảnh hưởng, owner, deadline, mitigation và bằng chứng khi đóng. Pipeline phải quét lại sau khi nâng dependency/image.

Bài tập quyền tối thiểu

Liệt kê ba identity: developer đọc log, App Service đọc đúng secret Key Vault và ghi Blob, pipeline triển khai vào một resource group. Với từng identity, ghi scope nhỏ nhất, hành động cần, cách cấp token và cách thu hồi. Kiểm tra riêng quyền control plane Azure RBAC và quyền data plane của dịch vụ đích; thấy resource trên portal không luôn đồng nghĩa đọc được dữ liệu bên trong.

AI hỗ trợ remediation: nhánh tùy chọn

Sau khi làm chủ pipeline, identity, logging và rollback, có thể thiết kế agent đọc ticket bảo mật, phân loại, thu bằng chứng, đề xuất thay đổi và tạo PR. Agent không nên tự đóng ticket hoặc sửa production chỉ vì mô hình trả lời tự tin. Ticket/tài liệu bên ngoài có thể chứa hướng dẫn độc hại; chỉ cấp quyền cần thiết, ghi audit, yêu cầu approval cho thay đổi rủi ro và dùng kiểm tra xác định để quyết định pass/fail.

Diễn tập trên giấy: ticket yêu cầu xoay secret. Viết luồng xác minh scope -> tìm nơi dùng -> lập kế hoạch -> PR -> test môi trường thấp -> approval -> rotation/rollout -> xác nhận request -> thu bằng chứng -> đóng ticket. Chỉ ra bước máy có thể làm, bước người duyệt, và phương án phục hồi nếu secret mới làm app lỗi. Đây là bài thiết kế, không cấp token cloud thật cho agent lab.

Case study có lời giải: Private Endpoint đã tạo nhưng SQL vẫn lỗi

Bối cảnh: nhóm vừa bật Private Endpoint cho Azure SQL; App Service báo kết nối timeout, portal vẫn cho thấy cả hai resource Running. Đi theo bốn lớp: (1) DNS từ môi trường chạy App Service đang phân giải FQDN SQL thành IP nào; (2) App Service có VNet integration/route tới private IP không; (3) NSG/firewall và dịch vụ đích có chấp nhận kết nối không; (4) token/DB user có quyền không. Nếu timeout trước TLS, sửa RBAC trước là sai hướng. Nếu TCP/TLS thành công nhưng SQL trả lỗi login, network có thể đã đúng; kiểm tra identity và quyền trong DB.

Bằng chứng kết thúc: truy vấn thử từ đúng runtime identity thành công, public path được cấu hình theo chính sách, telemetry ghi độ trễ và lỗi trở về baseline. Chỉ kiểm tra từ laptop kỹ sư không chứng minh App Service có cùng đường mạng. Ghi diagram DNS/route cuối cùng để người sau không tạo endpoint trùng hoặc mở lại public access khi gặp lỗi.

Cổng năng lực

Bạn vẽ được App Service/Function/Storage/SQL/Service Bus cùng identity/network; giải thích private endpoint + DNS; thiết kế pipeline có môi trường và quyền; so sánh Bicep/ARM/Terraform; đề xuất truy vấn và cảnh báo từ App Insights/KQL. Nếu chưa có Azure lab trả phí, ghi rõ phần nào mới học qua tài liệu và thiết kế.

Một ứng dụng Azure theo ba lớp: identity, network, runtime

Ứng dụng chạy trên App Service/Functions/AKS hay VM đều cần danh tính để đọc Key Vault, Storage, Service Bus, Azure SQL. **Managed identity** giảm việc lưu secret dài hạn; nhưng vẫn cần cấp role/quyền đúng resource và kiểm tra app thực sự dùng identity nào. **Entra ID** quản lý danh tính/ứng dụng; **Azure RBAC** kiểm soát nhiều hành động trên tài nguyên Azure theo scope; **data-plane permission** của từng dịch vụ có thể cần cấu hình riêng. Network VNet, NSG, firewall và Private Endpoint quyết định đường đi, không thay authorization. Một app có quyền đọc SQL nhưng DNS trỏ public IP vẫn có thể timeout; app tới private IP nhưng thiếu DB permission vẫn bị từ chối.

Chọn compute theo bài toán: App Service phù hợp web app với ít gánh quản trị host; Functions phù hợp event-driven/trigger và có đặc tính cold start/timeout/scaling theo plan; AKS cho orchestration và khả năng tùy biến nhưng tăng trách nhiệm cluster; VM cho kiểm soát OS và gánh patch/hardening lớn hơn.

Không chọn AKS chỉ vì JD có Kubernetes nếu hệ thống nhỏ đủ dùng App Service. Với Service Bus, cần hiểu queue/topic, delivery, retry, dead-letter, identity và giới hạn downstream. Storage có blob/file/queue/table tùy dịch vụ; chọn theo API và consistency/cost chứ không gọi chung “Storage”.

Private Endpoint: một đường DNS cụ thể

Giả sử Notes API trên App Service cần Azure SQL Database qua private endpoint. App vẫn dùng hostname SQL chuẩn trong connection string để TLS/name validation hoạt động. Từ **runtime của App Service**, DNS phải trả private IP phù hợp, VNet integration/route phải dẫn tới subnet endpoint, NSG/firewall/SQL connection policy phải cho traffic và SQL auth phải chấp nhận identity. Tạo Private Endpoint resource trong portal chỉ hoàn thành một mắt xích. Nếu laptop **nslookup** trả public IP nhưng app trong VNet trả private IP, điều đó có thể là thiết kế split DNS dự kiến. Tránh hardcode private IP vào connection string.

Đường điều tra: DNS từ app -> TCP tới host/port -> TLS/certificate -> auth Entra/SQL -> query/schema. Nếu DNS sai, đừng cấp thêm Owner role. Nếu **Login failed**, đừng mở firewall rộng. Nếu query chậm sau khi kết nối, nhìn pool/DB metrics và query plan. Azure SQL Database và SQL Managed Instance có chi tiết endpoint/DNS khác; đọc đúng tài liệu dịch vụ. Nguồn: Azure SQL Database private endpoint, Azure Private Endpoint.

Lab thiết kế không phát sinh chi phí

Vẽ staging Notes API dùng App Service, Azure SQL, Key Vault, Storage và Service Bus. Lập bảng cho mỗi kết nối: nguồn, đích, DNS, IP public/private, port/protocol, managed identity/role, secret/certificate, telemetry và một failure mode. **Bước 1:** viết hai request: người dùng gọi API và API gửi message. **Bước 2:** cho app quyền tối thiểu, CI quyền deploy staging nhưng không đọc DB production. **Bước 3:** mô phỏng DB private DNS sai: app thấy lỗi gì, KQL/App Insights ghi dấu hiệu nào, sửa ở DNS hay code? **Bước 4:** mô phỏng Service Bus consumer ngừng: backlog, age of oldest message, dead-letter tăng; alert nào hành động được? **Bước 5:** dùng calculator chính thức ước tính chi phí theo region/SKU hiện tại trước khi tạo tài nguyên thật.

Lời giải định hướng: không có một “network OK” chung. DNS/route/TCP/auth là bốn phép kiểm khác nhau. Key Vault secret reference/managed identity cần permission đúng; nếu app start trước khi permission propagation, cần log rõ và retry có giới hạn. Service Bus có thể nhận message trong khi consumer chết, nên API trả 202 không chứng minh công việc đã hoàn tất. Ghi correlation ID từ HTTP tới message và consumer.

Azure DevOps Pipelines và IaC

YAML pipeline định nghĩa trigger, stages/jobs/tasks, variables, environments và approvals theo cấu hình tổ chức. Bicep/ARM mô tả tài nguyên Azure; Terraform dùng state và provider. Dù dùng công cụ nào, plan/what-if cần review, identity pipeline cần scope hẹp và mỗi môi trường có gate. Nếu dùng service connection có quyền subscription Owner cho mọi PR, thay pipeline là đường tấn công. Tách CI build artifact khỏi CD/promotion, ghi digest và deployment revision; smoke test sau deploy và rollback có điều kiện với DB migration.

App Insights/Log Analytics/KQL giúp đọc request/dependency/exception nếu instrumentation và ingestion được cấu hình. Một query KQL hữu ích cần filter thời gian, app/operation ID, status và join/correlation với dependency; không tải mọi log rồi tìm bằng mắt. Cân nhắc sampling, retention, PII và chi phí ingestion. Nếu alert không có owner/runbook hoặc page mỗi lúc CPU nhấp nhô, nó sẽ bị bỏ qua.

Nhánh AI tự động hóa trong JD: chỉ sau nền tảng

Agent đọc ticket S360/SFI/IcM và đề xuất fix có thể tiết kiệm toil, nhưng agent cần quyền hẹp, bằng chứng nguồn, audit trail, dry-run, policy gate và human approval cho thay đổi rủi ro. Phân biệt **đề xuất** với **áp dụng** và **đóng ticket**. Một ticket "rotate secret" không đủ thông tin để agent tự xoay production key nếu không biết consumer nào, thời gian overlap, rollback và owner. Bắt đầu bằng phân loại/thu thập bằng chứng có người review, đo precision/false action, rồi mới mở rộng thao tác tự động. Đó là DevOps engineering, không phải chỉ gắn LLM API vào pipeline.

Câu hỏi có lời giải: managed identity có xóa nhu cầu RBAC không? Không; nó là danh tính, vẫn cần quyền. **Private Endpoint có tự sửa DNS không?** Phụ thuộc cấu hình DNS/link của môi trường; phải kiểm tra từ runtime. **App Insights xanh có chứng minh business flow?** Không nếu chưa instrument đúng giao dịch. Nguồn: Azure Well-Architected, Azure DevOps YAML schema.

Một lần deploy Azure theo bằng chứng

CI build image có digest và test; IaC plan/what-if mô tả thay đổi hạ tầng; reviewer duyệt identity/network/storage; pipeline deploy staging bằng service connection phạm vi staging; smoke test gọi API thật; production gate duyệt digest đã thử. Sau deploy, xem App Insights request failure/latency và dependency SQL/Service Bus, so operation ID với mốc release. Nếu rollback, xác định app image, config, schema DB và queue messages có còn tương thích không. Azure DevOps Pipelines YAML giúp mô tả luồng nhưng không tự thay quyết định rủi ro.

Case App Service không đọc Key Vault

App trả 500 ngay sau start vì không lấy được secret. Kiểm tra managed identity của đúng App Service slot/runtime, role/access policy trên đúng vault/secret, network/private endpoint của vault, DNS từ app và thời gian propagation. Một "Key Vault reference" trong config không chứng minh được phân giải ở runtime. Nếu staging slot có identity khác production slot, swap có thể đổi behavior. Giảm tác động bằng rollback config/slot theo runbook nếu tương thích; không copy secret vào plain app settings để chữa cháy mà không có kiểm soát.

Bài tập KQL trên giấy

Bạn có bảng request với timestamp, operation_Id, url, resultCode, duration và bảng dependency có operation_Id, target, duration, success. Hãy lọc 15 phút quanh release, nhóm 5xx theo route và ghép các operation lỗi với dependency SQL để xem correlation. Nếu app chưa truyền operation ID tới downstream, phải thêm instrumentation; KQL không thể tự tạo mối liên hệ không có trong telemetry. Không thu thập body/token nhạy cảm chỉ để debug tiện.

Câu hỏi: Functions trả 202 sau khi gửi Service Bus, đã hoàn tất nghiệp vụ chưa? Chưa; consumer có thể còn backlog hoặc dead-letter. Theo dõi end-to-end correlation và age. Nguồn: Azure Monitor/Application Insights, Azure DevOps Pipelines.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://learn.microsoft.com/azure/well-architected/
- https://learn.microsoft.com/azure/private-link/private-endpoint-overview
- https://learn.microsoft.com/azure/devops/pipelines/yaml-schema/?view=azure-pipelines